



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра прикладной математики

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа №1

Введение в инструментарий анализа данных

Студент группы ИНБО-12-23, Албахтин И.В.

(подпись)

Преподаватель Воронцов Ю.А.

(подпись)

Москва 2026 г.

Содержание

1	Цель и результаты работы	3
2	Окружение и воспроизводимость.....	3
3	Учебный набор FiresRu	3
3.1	Паспорт и качество данных.....	3
3.2	Типы и вычисляемые признаки	4
3.3	Фильтрация и групповые статистики.....	4
3.4	Корреляции и потенциальные выбросы	5
3.5	Визуальный анализ	5
3.6	Экспорт и проверка повторного чтения.....	7
4	Самостоятельная работа с Bike Sharing	8
4.1	Источник и паспорт данных.....	8
4.2	Диагностика и преобразования	8
4.3	Фильтрация и агрегации.....	9
4.4	Числовой анализ и выбросы	9
4.5	Четыре визуализации самостоятельной части	10
4.6	Экспорт самостоятельной части.....	12
5	Выводы и ограничения	12
	Практическая работа №1. Подробные ответы на контрольные вопросы	14
	Источники и приложенные материалы	21

1 Цель и результаты работы

Цель работы — выполнить полный цикл анализа табличных данных средствами NumPy и Pandas: диагностику, обоснованную предобработку, фильтрацию, агрегации, разведочный анализ и проверку экспортированных файлов. Учебная часть выполнена для FiresRu; самостоятельная — для дневного набора Bike Sharing. Обе части представлены в едином выполненном Jupyter Notebook.

FiresRu содержит 26 681 наблюдение и девять исходных полей; Bike Sharing — 731 наблюдение и 16 полей. Пропусков и полных дубликатов нет. Для каждого набора получены четыре графика и три экспорта; количество строк и значения после повторного чтения проверены.

2 Окружение и воспроизводимость

Notebook выполнен сверху вниз в чистом ядре отдельной виртуальной машины TIABD-Albakhtin-IV (Ubuntu 22.04.5, 2 vCPU, 4 ГБ RAM). Источники сохранены отдельно от очищенных таблиц и графиков. Версии библиотек выводятся в первой ячейке каждой части; в ноутбуке сохранены результаты всех 31 кодовых ячеек без ошибок. Случайные выборки для графиков используют `random_state=42`.

```
import platform, numpy as np, pandas as pd
print(platform.python_version(), np.__version__, pd.__version__)
df = pd.read_csv("FiresRu.csv", encoding="utf-8-sig")
df.head()
df.info(memory_usage="deep")
df.describe(include="all")
```

3 Учебный набор FiresRu

3.1 Паспорт и качество данных

Источник — предоставленный с заданием файл FiresRu.csv. Одна строка описывает категорию события, координаты и погодные характеристики. Даты и времени наблюдения в таблице нет; временную динамику по номеру строки получить нельзя. Для влажности и солнечной радиации единицы не

подтверждены отдельными метаданными и не дописываются по предположению.

Результаты приведены в таблице 1.

Таблица 1 – Поля исходного набора FiresRu

Поле	Содержание
type_id	Код категории события
type_name	Название категории
lon, lat	Географические координаты
temperature_c	Температура, °C
precipitation_mm	Осадки, мм
relative_humidity	Влажность; единица не подтверждена
wind_speed_ms	Скорость ветра, м/с
solar_radiation	Солнечная радиация; единица не подтверждена

Проверки `isna()`, `uplicated()`, `nunique()` и перекрёстная таблица `type_id/type_name` подтверждают отсутствие пропусков, полных повторов и противоречий между кодом и названием. Числовой `type_id` трактуется как категория. Строки без основания не удалялись.

3.2 Типы и вычисляемые признаки

```
clean_df = df.copy()
clean_df["type_name"] = clean_df.type_name.astype("category")
clean_df["type_id"] = pd.to_numeric(clean_df.type_id, downcast="integer")
clean_df["wind_speed_kmh"] = (clean_df.wind_speed_ms * 3.6).round(2)
clean_df["is_forest_fire"] = clean_df.type_name.eq("Forest fire")
```

Память исходной таблицы — 3 544 759 байт; после оптимизации типов — 1 548 147 байт. Снижение составляет 56,33%. Сравнение выполнено на одинаковых девяти исходных столбцах до добавления вычисляемых полей. Затем созданы скорость ветра в км/ч и признак лесного пожара.

3.3 Фильтрация и групповые статистики

```
subset = df.loc[(df.relative_humidity < 40) & (df.wind_speed_ms > 2)]
assert subset.index.equals(df.query("relative_humidity < 40 and wind_speed_ms > 2").index)
type_stats = df.type_name.value_counts()
group_stats = clean_df.groupby("type_name", observed=True).agg(
    n=("type_name", "size"), temp_mean=("temperature_c", "mean"),
    humidity_mean=("relative_humidity", "mean"), wind_mean=("wind_speed_ms", "mean"))
```

Составной фильтр отобрал 337 строк (1,26%). Пороги используются для учебной демонстрации условия и не объявляются нормативом пожарной

опасности. Выполнены частотная группировка и агрегация пяти погодных характеристик по категории события.

Результаты приведены в таблице 2.

Таблица 2 – Сравнение категорий событий

Категория	Строк	Доля %	Средняя Т °С	Ветер м/с
Forest fire	18 415	69,02	18,69	0,42
Natural fire	5 195	19,47	17,42	0,80
Uncontrolled burn	2 711	10,16	14,21	1,64
Controlled burn	333	1,25	10,78	1,46
Peat fire	27	0,10	18,17	1,19

3.4 Корреляции и потенциальные выбросы

Корреляция температуры и влажности по всем строкам составляет - 0,155; температуры и ветра — -0,297. Слабая линейная связь не исключает нелинейной зависимости и не показывает причинности.

Для ветра $Q1=0,110$, $Q3=0,660$, $IQR=0,550$. Правило $[Q1-1,5 \cdot IQR; Q3+1,5 \cdot IQR]$ выделило 3 652 кандидата на выбросы. Они сохранены: редкое или асимметричное значение без предметной проверки не считается ошибкой.

3.5 Визуальный анализ

На рисунке 1 представлен результат анализа.

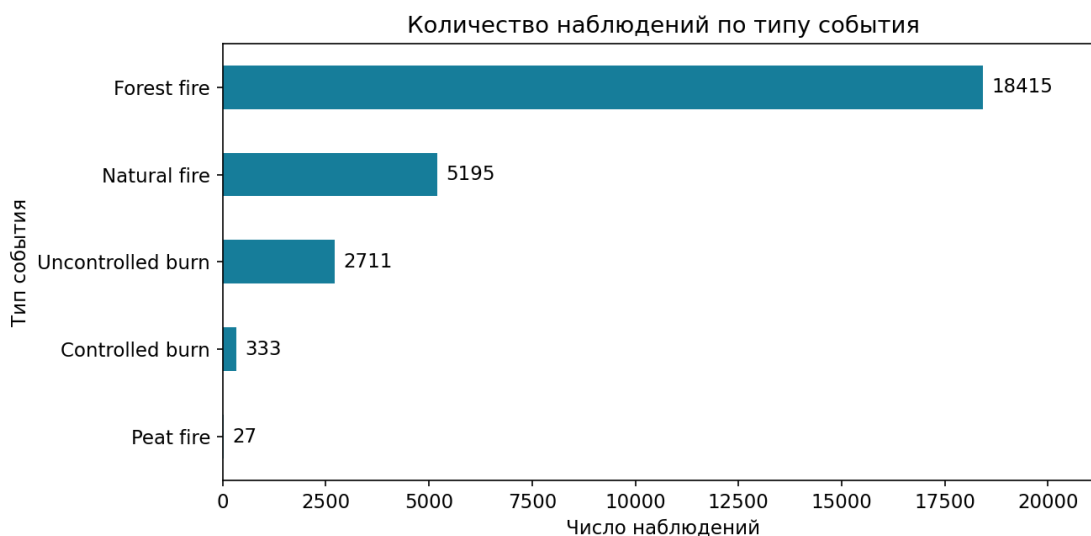


Рисунок 1 – Число наблюдений по категориям событий

Forest fire занимает 69,02% наблюдений; Peat fire представлен 27 строками. Средние малых групп требуют осторожной интерпретации.

На рисунке 2 представлен результат анализа.

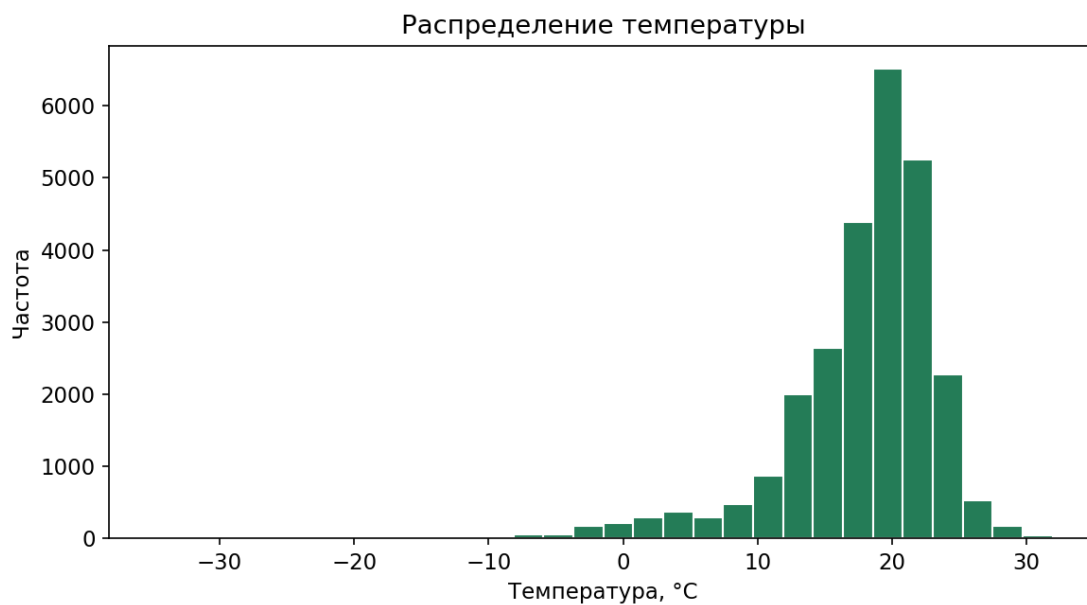


Рисунок 2 – Распределение температуры

Основная масса температур положительна; левый хвост содержит отрицательные значения. Без сведений о времени и источнике их нельзя автоматически исключать.

На рисунке 3 представлен результат анализа.



Рисунок 3 – Температура и влажность в выборке из 5 000 строк

Широкое облако точек согласуется со слабой линейной связью. Для коэффициента используются все строки, для рисунка — воспроизводимая выборка.

На рисунке 4 представлен результат анализа.

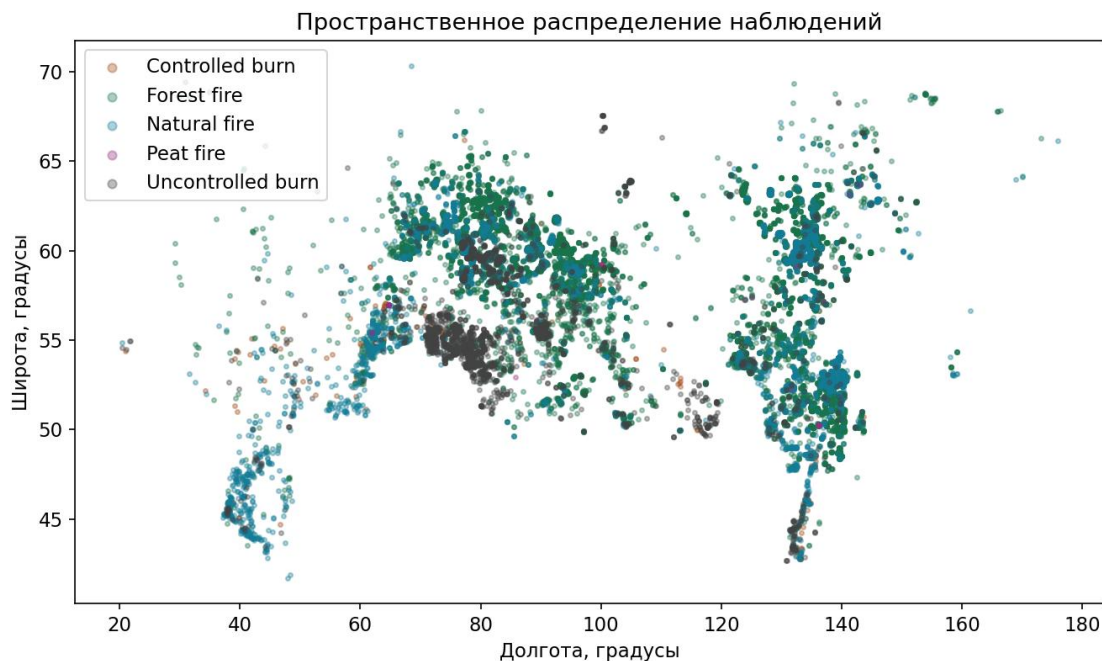


Рисунок 4 – Пространственное расположение событий

Координаты показывают неоднородное расположение наблюдений. График не позволяет сравнить риски регионов без площади, населения и полноты регистрации.

3.6 Экспорт и проверка повторного чтения

Результаты приведены в таблице 3.

Таблица 3 – Экспорт очищенного FiresRu

Формат	Байт	Чтение мс	Размер	Значения
csv	1 800 593	6,414	26681 × 11	Совпали
json	7 183 440	67,292	26681 × 11	Совпали
parquet	647 244	2,189	26681 × 11	Совпали

CSV и JSON требуют восстановления типов при чтении. Parquet хранит схему и обеспечивает выборочное чтение столбцов. В данном сохранённом запуске он занял меньше места и прочитан быстрее; это наблюдение об этих данных, а не универсальная оценка форматов.

4 Самостоятельная работа с Bike Sharing

4.1 Источник и паспорт данных

Bike Sharing — дневные наблюдения Capital Bikeshare в Вашингтоне за 2011–2012 годы. Автор набора — Hadi Fanaee-T; источник UCI Machine Learning Repository, DOI 10.24432/C5W894, лицензия CC BY 4.0. Файл `day.csv` содержит 731 строку и 16 столбцов, включая дату, погодные показатели, категориальные коды и число прокатов. Требования о 500 строках, пяти информативных полях, двух числовых признаках и категориальном признаке выполнены.

Результаты приведены в таблице 4.

Таблица 4 – Группы признаков Bike Sharing

Поля	Содержание
<code>dteday, yr, mnth, weekday</code>	Дата и календарные признаки
<code>season, holiday, workingday, weathersit</code>	Категориальные коды сезона, праздника, рабочего дня и погоды
<code>temp, atemp, hum, windspeed</code>	Нормированные погодные показатели
<code>casual, registered, cnt</code>	Прокаты по типу пользователя и общий итог
<code>instant</code>	Технический индекс; исключён из содержательного анализа

4.2 Диагностика и преобразования

Проверены `head()`, `info()`, `describe(include="all")`, пропуски, уникальные значения, полные повторы, повторные даты и идентификаторы. Даты уникальны; год, месяц и день недели согласуются с `dteday`; отрицательных счётчиков нет; `cnt=casual+registered` для всех строк. Нормированные погодные показатели находятся в диапазоне `[0; 1]`. Нулевая влажность обнаружена в одной строке и сохранена с ограничением: это числовое значение, а не `NULL`.

Дата преобразована в `datetime`, календарные и погодные коды — в `category`, бинарные признаки — в `bool`, счётчики — в подходящие меньшие целочисленные типы. Для `season` в разных описаниях встречается различная словесная расшифровка; анализ использует исходные коды. Температура остаётся нормированной, поскольку обратный перевод в °C по противоречивым описаниям создавал бы неопределённость.


```
clean_df["dteday"] = pd.to_datetime(df.dteday, format="%Y-%m-%d")
clean_df["humidity_pct"] = clean_df.hum * 100
clean_df["casual_share_pct"] = clean_df.casual / clean_df.cnt * 100
clean_df["humidity_zero_flag"] = clean_df.hum.eq(0)
```

Память до оптимизации — 136 825 байт, после оптимизации — 41 545 байт; уменьшение 69,64%. Сравнение выполнено на одинаковых 16 исходных столбцах. Затем добавлены влажность в процентах, доля незарегистрированных пользователей и флаг нулевой влажности. Доля casual сравнивает структуру спроса в дни с различным общим количеством поездок.

4.3 Фильтрация и агрегации

Составной фильтр `workingday==True, temp>0,5` и `humidity_pct<65` отобрал 125 дней. Условия отражают рабочие дни с повышенной нормированной температурой и влажностью ниже 65%. Выполнены группировки по погоде, типу дня, году и календарному месяцу. Группировка по погоде включает `size, mean, median, std` и `sum`.

Результаты приведены в таблице 5.

Таблица 5 – Число прокатов в разных погодных категориях

Код погоды	Дней	Среднее	Медиана	Итого
1	463	4 876,79	4 844	2 257 952
2	247	4 035,86	4 040	996 858
3	21	1 803,29	1 817	37 869

В ясной погодной категории средний спрос выше, чем в категориях облачности и небольших осадков. Группы имеют разный размер; это наблюдаемое сопоставление, а не доказательство отдельного причинного эффекта погоды.

Результаты приведены в таблице 6.

Таблица 6 – Годовая динамика Bike Sharing

Год	Дней	Всего прокатов	В среднем в день
2011	365	1 243 103	3 405,76
2012	366	2 049 576	5 599,93

4.4 Числовой анализ и выбросы

Корреляция нормированной температуры с количеством прокатов составляет 0,627. Между `temp` и `atemp` наблюдается близкая к единице связь,

поэтому эти признаки несут во многом сходную информацию. Полное число прокатов за период — 3 292 679; медиана — 4 548, среднее — 4 504,35 в день.

Результаты приведены в таблице 7.

Таблица 7 – Проверка числовых показателей по IQR

Признак	Q1	Q3	Кандидатов
cnt	3 152,000	5 956,000	0
temp	0,337	0,655	0
humidity_pct	52,000	73,021	2
windspeed	0,135	0,233	13

Для cnt и temp кандидатов по IQR нет; для влажности найдено два, для ветра — 13. Значения сохранены и не интерполируются без проверки первичного источника. Отсутствие NULL не исключает смысловую аномалию, как показывает нулевая влажность.

4.5 Четыре визуализации самостоятельной части

На рисунке 5 представлен результат анализа.

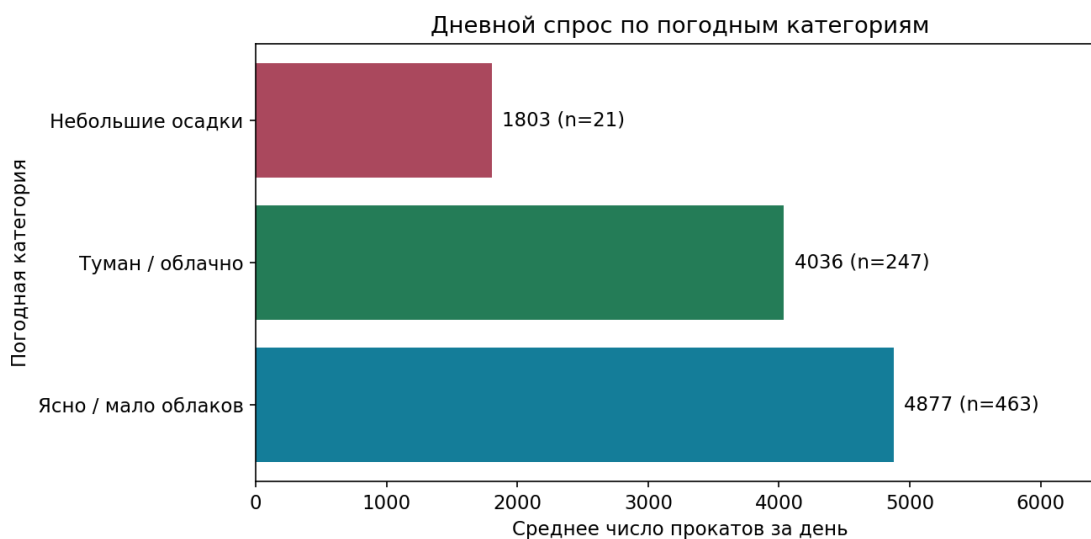


Рисунок 5 – Среднее число прокатов по погодным категориям

Сравнение категорий дополняет таблицу групповых средних; категория осадков содержит только 21 день.

На рисунке 6 представлен результат анализа.

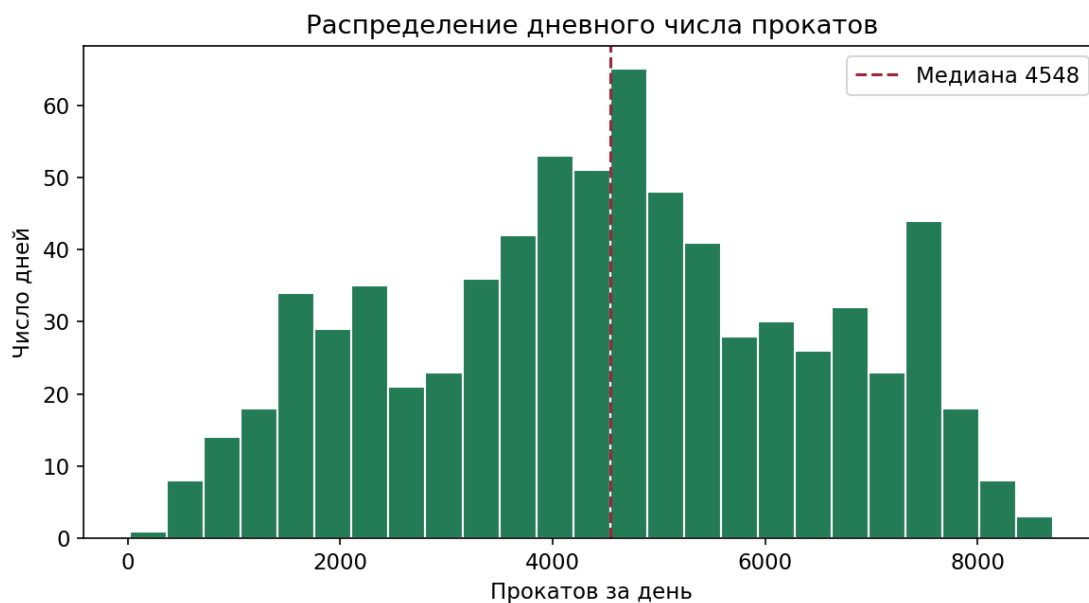


Рисунок 6 – Распределение дневного спроса

Дневной спрос существенно меняется. Одного среднего недостаточно; нужны медиана, распределение и календарные срезы.

На рисунке 7 представлен результат анализа.

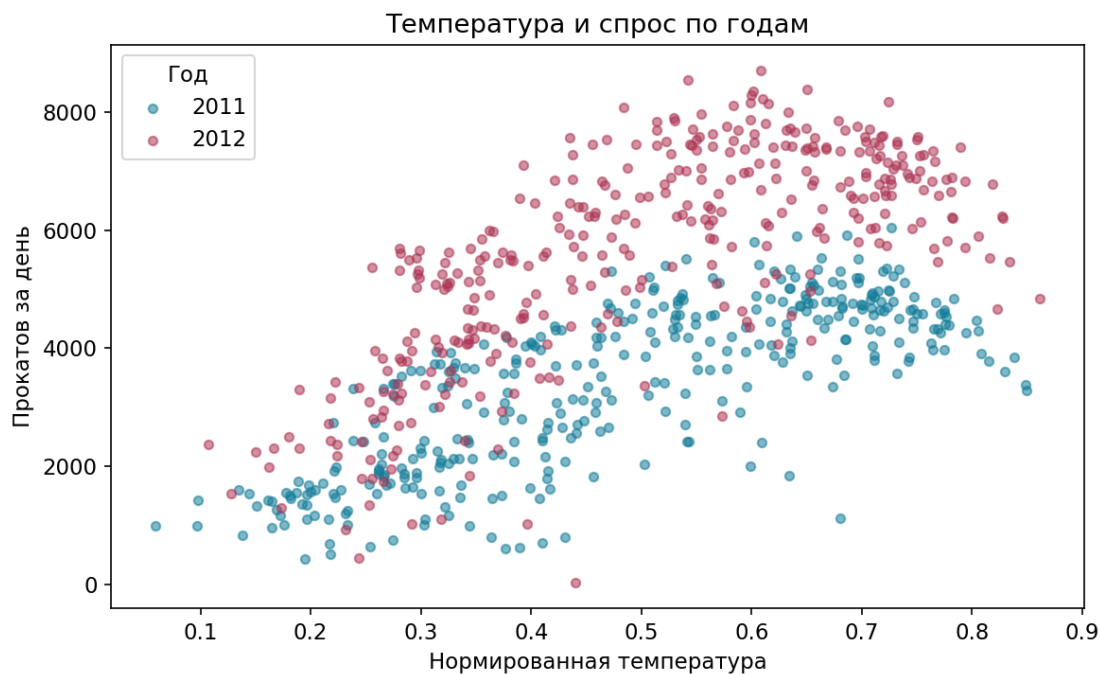


Рисунок 7 – Температура и число прокатов

Рост спроса связан с увеличением нормированной температуры, но разброс отражает влияние календаря, времени и других факторов.

На рисунке 8 представлен результат анализа.



Рисунок 8 – Средний дневной спрос по календарным месяцам

Среднесуточный максимум — сентябрь 2012 года: 7 285,77 проката в день, 218 573 за весь месяц. Сравнение одноимённых месяцев отделяет сезонность от общего роста системы.

4.6 Экспорт самостоятельной части

Результаты приведены в таблице 8.

Таблица 8 – Экспорт очищенного Bike Sharing

Формат	Байт	Чтение мс	Размер	Значения
csv	87 650	1,869	731 × 19	Совпали
json	251 341	6,295	731 × 19	Совпали
parquet	63 752	1,365	731 × 19	Совпали

Все три формата после чтения сохранили 731 строку, 19 столбцов и значения. Типы не совпали полностью без приведения: в частности, категории потребовали восстановления по сохранённой схеме BikeSharing_schema.json. Проверка dtypes выполняется отдельно от проверки значений; факт чтения файла сам по себе не подтверждает равенство типов.

5 Выводы и ограничения

1. Учебный набор FiresRu содержит пять резко несбалансированных категорий; Forest fire доминирует, а выводы о Peat fire ограничены 27 наблюдениями.

2. Обоснованные категориальные и целочисленные типы сократили память исходных столбцов FiresRu на 56,33%, Bike Sharing — на 69,64% при сохранении всех значений. Вычисляемые признаки добавлены после сравнения памяти.

3. Bike Sharing покрывает 731 последовательный день. Пропуски, полные дубликаты и нарушения равенства $\text{cnt}=\text{casual}+\text{registered}$ не найдены.

4. В погодной категории 1 среднее равно 4 876,79 проката; в категории 3 — 1 803,29. Погода связана со спросом, но группы существенно различаются по численности.

5. Средний дневной спрос вырос с 3 405,76 в 2011 году до 5 599,93 в 2012 году — примерно на 64,43%. Сравнивается среднее за день с учётом високосного года.

6. Месячный максимум — сентябрь 2012 года, 218 573 проката. Для анализа по месяцам использована реальная дата `dteday`.

7. Корреляция `temp` и `cnt` составляет 0,627. Это линейная связь в наблюдаемом наборе, а не причинная модель или прогноз.

8. Проверка IQR выявила аномальные кандидаты влажности и ветра; их автоматическое удаление изменило бы состав наблюдений без доказательства ошибки.

9. CSV, JSON и Parquet сохранили значения при повторном чтении. Parquet оказался компактнее в сохранённом запуске; категории проверены и восстановлены отдельно.

Ограничения FiresRu: отсутствует время, не подтверждены некоторые единицы и полнота регистрации событий. По координатам нельзя оценивать риск региона без внешних данных о территории и охвате наблюдений.

Ограничения Bike Sharing: только один город и два года; рост инфраструктуры и календарь могут смешиваться с погодой. Нулевая влажность и неоднозначная расшифровка `season` ограничивают предметную интерпретацию. IQR и корреляция не заменяют проверку первоисточника.

Практическая работа №1. Подробные ответы на контрольные вопросы

1. Почему числовой dtype не гарантирует, что признак имеет количественный смысл?

Простыми словами. dtype сообщает компьютеру, как значение хранится в памяти: как целое число, число с дробной частью, строка и так далее. Но компьютер не знает предметного смысла столбца. Целое число может быть настоящим количеством, например числом поездок, а может быть кодом категории, номером строки или идентификатором объекта.

Как это работает. Для количественного признака осмысленны арифметические действия и сравнение расстояний. Например, температура 20 °C действительно на 10 градусов больше температуры 10 °C. Для кода категории это неверно: код 4 не означает, что категория в два раза больше категории 2. Среднее, сумма и разность таких кодов обычно не имеют предметного смысла, хотя Pandas технически позволит их вычислить.

Пример и вывод. В FiresRu поле type_id хранится как целое число, но обозначает тип события. Значение 3 означает Forest fire, а 1 — Uncontrolled burn. Нельзя сказать, что лесной пожар «в три раза больше» неконтролируемого выжигания только из-за кодов. Поэтому при анализе надо смотреть не только на dtype, но и на описание поля, уникальные значения и связь кода с реальным объектом.

2. Чем DataFrame отличается от NumPy ndarray?

Простыми словами. NumPy ndarray — это прежде всего многомерная сетка значений. Pandas DataFrame — таблица, похожая на лист Excel: у неё есть имена столбцов, индекс строк и отдельный тип данных для каждого столбца. DataFrame построен поверх NumPy и других массивов, но добавляет удобные средства именно для табличного анализа.

Как это работает. В обычном ndarray все элементы имеют один общий dtype. Если смешать числа и строки, NumPy часто приведёт всё к одному совместимому типу, например к строкам или объектам. В DataFrame один

столбец может содержать даты, второй — категории, третий — целые числа, четвёртый — дробные значения. К столбцам обращаются по именам, данные можно соединять по индексам, группировать, фильтровать и агрегировать.

Пример и вывод. В таблице Bike Sharing столбец `dteday` является датой, `season` — категорией, а `cnt` — числом поездок. `DataFrame` сохраняет эту схему и позволяет написать `df.groupby("season")["cnt"].sum()`. В NumPy пришлось бы отдельно помнить номера колонок и вручную следить за типами. NumPy удобнее для плотных однородных числовых вычислений, `DataFrame` — для данных, организованных как таблица.

3. Что проверяют `head()`, `info()` и `describe()` и почему одной функции недостаточно?

Простыми словами. Эти функции отвечают на разные вопросы. `head()` показывает, как выглядят несколько реальных строк. `info()` описывает устройство таблицы. `describe()` рассчитывает краткую статистику по значениям. Вместе они дают первый обзор данных, но каждая функция видит только свою сторону набора.

Как это работает. `head()` помогает заметить названия столбцов, формат дат, разделители и очевидные странности в первых строках. `info()` показывает число строк и столбцов, количество непустых значений, распознанные типы и примерный расход памяти. `describe()` для числовых признаков выводит количество, среднее, стандартное отклонение, минимум, квартили и максимум; с `include="all"` она также описывает категориальные поля.

Пример и вывод. Если `head()` показывает пять корректных строк, это не доказывает, что остальные тысячи строк тоже корректны. `info()` может показать, что пропусков нет, но не заметит невозможную влажность -20. `describe()` обнаружит необычный минимум, но не покажет исходную строку целиком. Поэтому первичная диагностика обычно включает все три функции, а затем проверки пропусков, дубликатов, диапазонов и уникальных значений.

4. Почему отсутствие пропусков и дубликатов не гарантирует хорошее качество данных?

Простыми словами. Пропуск — это только один вид проблемы, а полный дубликат — ещё один. Таблица может быть полностью заполнена и не содержать одинаковых строк, но при этом хранить неверные значения, несогласованные единицы измерения или данные, которые плохо представляют исследуемую ситуацию.

Как это работает. К проблемам качества относятся невозможные диапазоны, опечатки в категориях, разные форматы дат, перепутанные единицы, ошибочные измерения, устаревшие записи и смещение выборки. Бывают и логические противоречия между столбцами. Например, итоговое число поездок должно равняться сумме случайных и зарегистрированных пользователей. Если равенство нарушено, данные неконсистентны, даже если ни одной пустой ячейки нет.

Пример и вывод. В Bike Sharing нулевая влажность записана обычным числом, поэтому `isna()` её не обнаружит. Такое значение надо проверять по документации и источнику. В FiresRu код события и его название должны образовывать однозначную пару; иначе возникнет смысловая ошибка. Проверка качества должна охватывать полноту, уникальность, допустимые диапазоны, единицы, связи между полями и происхождение данных.

5. Когда тип `category` уменьшает расход памяти и почему?

Простыми словами. Строковый столбец обычно хранит много повторяющихся текстовых объектов. Тип `category` заменяет каждый повторяющийся текст небольшим числовым кодом, а полный текст сохраняет один раз в отдельном словаре категорий.

Как это работает. Предположим, в таблице миллион строк, но в столбце только значения Москва, Казань и Омск. Вместо миллиона длинных строк Pandas хранит три названия и миллион компактных кодов 0, 1 или 2. Чем меньше уникальных значений относительно числа строк и чем длиннее

повторяющийся текст, тем заметнее экономия. Если почти каждая строка уникальна, словарь категорий и коды могут не дать выигрыша.

Пример и вывод. В FiresRu столбец `type_name` содержит всего пять названий на 26 681 строку, поэтому `category` подходит хорошо. После преобразования расход памяти таблицы снизился примерно на 56,33%. Перед преобразованием полезно сравнить `nunique()` с числом строк и затем измерить память через `memory_usage(deep=True)` — тип нельзя выбирать только по привычке.

6. Чем фильтрация отличается от группировки?

Простыми словами. Фильтрация отвечает на вопрос «какие конкретные строки подходят под условие?». Группировка отвечает на вопрос «какие сводные показатели получаются для каждой категории или группы?». Это разные операции и часто они используются последовательно.

Как это работает. При фильтрации создаётся логическая маска из `True` и `False`. Pandas оставляет строки, где условие истинно: `df.loc[df["wind_speed_ms"] > 2]`. При группировке `groupby()` делит строки по значению ключа, после чего к каждой части применяют функцию: `count`, `sum`, `mean`, `median` и другие. Результат группировки обычно содержит меньше строк, потому что каждая группа превращается в сводку.

Пример и вывод. Условие «влажность меньше 40 и ветер сильнее 2 м/с» выбирает 337 отдельных наблюдений FiresRu — это фильтрация. Таблица со средними температурой и ветром для каждого `type_name` — группировка. Можно сначала отфильтровать только нужный регион или период, а затем сгруппировать оставшиеся строки.

7. Что означает коэффициент корреляции около нуля и какие выводы из него делать нельзя?

Простыми словами. Корреляция Пирсона измеряет силу линейной связи двух числовых признаков. Значение около 1 соответствует сильной прямой

линейной связи, около -1 — сильной обратной, а около 0 означает, что выраженной прямой линии между признаками в этой выборке не видно.

Как это работает. Корреляция около нуля не доказывает отсутствие любой связи. Зависимость может быть криволинейной: например, $y = x^2$ симметрична относительно нуля и может дать слабую линейную корреляцию. Связь также может различаться по группам, скрываться из-за выбросов или зависеть от третьего признака. И наоборот, высокая корреляция не доказывает, что один признак вызывает другой.

Пример и вывод. В FiresRu наибольшие по модулю корреляции между рассмотренными погодными признаками находятся примерно на уровне $0,30$. Это говорит лишь о слабой линейной сопряжённости в данном наборе. Следующий шаг — посмотреть scatter plot, проверить группы и выбросы, а не писать «связи нет» или «один показатель влияет на другой».

8. Почему правило IQR нельзя использовать как автоматический алгоритм удаления данных?

Простыми словами. IQR — межквартильный размах, то есть расстояние между первым квартилем $Q1$ и третьим квартилем $Q3$. Правило помечает значения ниже $Q1 - 1,5 \cdot IQR$ и выше $Q3 + 1,5 \cdot IQR$ как необычные. Оно находит кандидатов для проверки, но не умеет определять, является ли значение ошибкой.

Как это работает. В распределении с длинным естественным хвостом много корректных наблюдений окажутся за границами IQR. Например, редкие сильные порывы ветра могут быть реальными. Удаление таких строк меняет средние, дисперсию, корреляции и доли категорий. Решение требует знания предметной области, единиц измерения, условий сбора и цели анализа.

Пример и вывод. Для `wind_speed_ms` в FiresRu правило отмечает 3 652 строки, то есть около $13,69\%$ набора. Это слишком большая доля для бездумного удаления. Нужно проверить максимум, распределение, источник и возможные ограничения датчика. Корректные варианты — оставить значения,

преобразовать шкалу, использовать устойчивые методы или удалить только подтверждённые ошибки с объяснением причины.

9. Как выбрать между histogram, bar chart, scatter plot и временным рядом?

Простыми словами. Тип графика выбирают по вопросу и типам данных. Гистограмма показывает форму распределения одного числового признака. Столбчатая диаграмма сравнивает категории. Scatter plot показывает совместное поведение двух числовых признаков. Временной ряд показывает изменение показателя по настоящей временной оси.

Как это работает. Для гистограммы числовую шкалу делят на интервалы и считают наблюдения в каждом. В bar chart каждому отдельному названию категории соответствует столбец. В scatter plot каждая строка становится точкой с координатами x и y . Во временном ряду наблюдения сортируют по дате и при необходимости агрегируют по дням, месяцам или годам.

Пример и вывод. Распределение температуры FiresRu показано гистограммой, число событий разных типов — горизонтальными столбцами, связь температуры и влажности — scatter plot. Для Bike Sharing месячное число поездок строится как временной ряд по `dteday`. Нельзя выбирать график только по внешнему виду: он должен отвечать на конкретный вопрос без искажения структуры данных.

10. Почему индекс строк нельзя использовать как время, если он не является датой?

Простыми словами. Индекс 0, 1, 2, ... обычно означает только положение строки в таблице. Он не сообщает, когда произошло событие и сколько времени прошло между соседними строками. Поэтому линия по такому индексу показывает порядок хранения, а не изменение во времени.

Как это работает. После сортировки, удаления строк или объединения таблиц порядок индексов может измениться. Даже если записи случайно расположены хронологически, расстояние между индексами всегда равно

единице, тогда как между реальными событиями могли пройти минуты, дни или месяцы. Для временного анализа нужен столбец даты/времени, преобразованный через `pd.to_datetime()`, проверенный и отсортированный.

Пример и вывод. В FiresRu нет временного признака, поэтому восстановить месячную или годовую динамику по этому файлу нельзя. В Bike Sharing есть `dteday`, и месячная агрегация выполняется именно по нему. Если даты отсутствуют, следует прямо указать ограничение и выбрать другой график, а не выдавать номер строки за время.

11. Какие преимущества Parquet даёт аналитическому pipeline по сравнению с CSV?

Простыми словами. CSV — текстовый файл: каждая строка содержит значения, разделённые запятыми или другим символом. Parquet — бинарный колоночный формат, созданный для хранения и чтения больших аналитических таблиц. Его нельзя удобно открыть в обычном текстовом редакторе, зато программы обрабатывают его эффективнее.

Как это работает. Parquet хранит схему типов, поддерживает сжатие и располагает значения одного столбца рядом. Поэтому система может прочитать только нужные колонки, не разбирая весь файл. CSV не хранит точные типы: при каждом чтении программа заново угадывает числа, даты и строки. Также в CSV нужно отдельно учитывать разделитель, кодировку и десятичный знак.

Пример и вывод. После сохранения FiresRu полезно прочитать Parquet обратно и сравнить `shape` и `dtypes` с исходной таблицей. Parquet удобен для повторной аналитики и Spark, а CSV — для обмена с простыми программами. Размер и скорость зависят от конкретных данных и сжатия, поэтому их измеряют, а не обещают заранее.

12. Зачем выполнять Restart Kernel и Run All?

Простыми словами. Ядро Jupyter хранит переменные, импортированные библиотеки и промежуточные результаты в памяти. Если запускать ячейки в

случайном порядке, notebook может работать только потому, что нужная переменная осталась от старого запуска. После передачи файла другому человеку такая скрытая зависимость приведёт к ошибке.

Как это работает. Restart Kernel очищает память и создаёт новое состояние Python. Run All выполняет ячейки сверху вниз так, как их увидит проверяющий. Проверка обнаруживает использование переменной до её создания, зависимость от вручную загруженного файла, случайный результат без random_state, неверный рабочий каталог и отсутствующую библиотеку.

Пример и вывод. В этой работе Исходники/execute_all.py выполняет notebook в новом ядре python3 личной VM. После каждой ячейки сохраняется текущий результат; успешное завершение всех ячеек отдельно подтверждено в Результаты/execution.json. Воспроизводимый notebook должен запускаться командой Run All без ручных исправлений, потому что именно это доказывает полноту кода и правильный порядок шагов.

Источники и приложенные материалы

1. Методические материалы практической работы №1 и предоставленный FiresRu.csv.
2. H. Fanaee-T. Bike Sharing. UCI Machine Learning Repository. <https://doi.org/10.24432/C5W894>. CC BY 4.0.
3. Ноутбуки/ТИАБД_ПР1_АлбахтинИВ.ipynb: 31 выполненная кодовая ячейка, восемь графиков и проверки повторного чтения.
4. Результаты/ПР1/fires и Результаты/ПР1/bike_sharing: очищенные таблицы, групповые статистики, графики и JSON-сводки.